

AI Career Mentor Project Build Guide

A production-focused roadmap for building with OpenAI Codex

Product name: TBD

Document role: project-root execution guide

Build the smallest useful product first. Make every milestone testable. Let Codex accelerate execution, not replace engineering judgment.

1. Purpose of This Guide

This document is designed to live in the repository root and act as the shared source of truth for you, Codex, and future contributors. It defines the product boundary, architecture, execution order, quality gates, and working conventions.

- Codex is the coding agent used to accelerate implementation; it is not the product name.
- The product is a personalized AI career and software-engineering mentor.
- The temporary project label is AI Career Mentor until a final name is selected.
- The first release must focus on chat, interview practice, communication simulations, memory, and measurable feedback.

2. Product Vision

Build an AI mentor that helps a user learn, communicate, practise interviews, collaborate in realistic engineering scenarios, and grow toward a target role. The assistant should become more useful over time by remembering confirmed goals, projects, strengths, and recurring weaknesses.

Core user outcomes

- Explain technical concepts at the user's current level.
- Conduct realistic technical and behavioral interviews.
- Simulate stand-ups, group discussions, code reviews, and manager conversations.
- Guide users through building real software projects without hiding the reasoning.
- Track performance and recommend the next best practice activity.
- Use the user's approved memories and learning materials to personalize coaching.

3. MVP Boundary

The MVP is complete when a user can create an account, complete onboarding, start a persistent streamed conversation, run a structured mock interview, receive a scored report, and return later to find that confirmed goals and weak areas are remembered.

Build in MVP	Delay until later
Authentication and onboarding	Voice and real-time audio
Persistent streamed chat	Multiple LLM providers
Mentor mode	Autonomous multi-agent system
Technical and behavioral interviews	Microservices and Kubernetes
Structured scoring and reports	GitHub repository ingestion
Three workplace simulations	Browser code execution
Basic user-controlled memory	Fine-tuning and custom model training
Simple progress dashboard	Complex subscriptions and gamification

4. Recommended Stack

Layer	Choice	Reason
Web	Next.js + TypeScript	Modern full-stack UI, routing, streaming support,

Layer	Choice	Reason
		deployability.
UI	Tailwind CSS + shadcn/ui	Fast production-quality interface with accessible components.
API	FastAPI + Python	Typed asynchronous API and strong AI ecosystem.
Validation	Pydantic + Zod	Validated contracts on both backend and frontend.
Database	PostgreSQL	Reliable relational source of truth.
Vector search	pgvector	Keeps semantic memory and RAG inside PostgreSQL.
Auth	Supabase Auth	Reduces service count and integrates with PostgreSQL.
AI	OpenAI API behind a provider interface	Fast initial delivery without locking application logic to raw provider calls.
Streaming	Server-Sent Events	Simpler than WebSockets for one-way token streaming.
Testing	Pytest + Vitest + Playwright	Unit, integration, and critical browser flow coverage.
Monitoring	Sentry + structured logs	Error visibility and production debugging.
Deployment	Vercel + managed Python host + Supabase	Simple first production topology.

5. Architecture Decisions

- Start as a modular monolith, not microservices.
- Use one conversation orchestrator with mode-specific strategies, not many autonomous agents.
- Separate modes from personas: a mode controls behavior; a persona controls tone and role.
- Store messages as individual records, not one large conversation JSON object.
- Make saved memories visible, editable, and deletable by the user.
- Keep AI provider calls behind an internal interface.
- Use structured model outputs for interview scoring and memory extraction.
- Add Redis only after a demonstrated need for distributed state, queues, or rate limiting.

```

Browser / Next.js
  |
  | HTTPS + SSE
  v
FastAPI modular monolith
- auth
- conversations
- interviews
- simulations
- memory
- analytics
- AI provider layer
  |

```

V	V
PostgreSQL/pgvector	OpenAI API

6. Repository Layout

```
project-root/
|-- AGENTS.md
|-- PROJECT_GUIDE.pdf
|-- README.md
|-- docker-compose.yml
|-- Makefile
|-- apps/
|   |-- web/
|   |-- api/
|-- packages/
|   |-- contracts/
|   |-- prompts/
|-- docs/
|   |-- architecture/
|   |-- decisions/
|   |-- product/
|-- evals/
|-- infrastructure/
|-- .github/workflows/
```

7. How to Use Codex on This Project

Codex should operate from explicit repository instructions, small scoped tasks, and verifiable acceptance criteria. OpenAI documents that Codex reads AGENTS.md before working and supports layered project-specific guidance. Use this PDF for product and architecture context, while AGENTS.md contains concise operational rules Codex can apply every time.

Recommended AGENTS.md responsibilities

- State the project purpose and current milestone.
- List commands for setup, linting, type checking, testing, and database migrations.
- Define folder ownership and architectural boundaries.
- Require tests for changed behavior.
- Require Codex to inspect existing code before creating new abstractions.
- Ban secrets, destructive migrations, and unapproved dependency changes.
- Require a final summary containing files changed, tests run, and remaining risks.

```
# Example task given to Codex
Implement conversation creation for Milestone 2.
```

Scope:

- Add conversation model and migration.
- Add POST /api/v1/conversations.
- Add ownership checks.
- Add unit and integration tests.

Constraints:

- Follow existing repository patterns.
- Do not add new dependencies.
- Do not modify interview modules.

Acceptance criteria:

- Authenticated user can create a conversation.
- Anonymous request returns 401.
- Created conversation belongs to current user.
- Lint, type checks, and relevant tests pass.

Codex execution loop

1. Read AGENTS.md, this guide, and relevant existing files.
2. Restate the task boundary and identify affected modules.
3. Implement the smallest coherent change.
4. Run focused tests first, then the repository quality checks.
5. Review the diff for accidental changes, security issues, and missing error states.
6. Report exactly what changed, what was tested, and any unresolved risk.

8. Core Data Model

Entity	Purpose
profiles	user identity context, target role, level, stack, communication goals
conversations	mode, persona, status, title, metadata
messages	role, content, model, tokens, latency, metadata
interview_sessions	type, difficulty, status, score, timestamps
interview_questions	question, answer, follow-up, competencies
evaluations	correctness, clarity, depth, confidence, strengths, weaknesses
memory_records	category, content, confidence, importance, embedding, source
goals	title, type, status, target date, progress
usage_records	provider, model, tokens, cost estimate, feature, timestamp

9. Milestone Roadmap

Milestone	Deliverable	Exit gate
M0 - Foundation	Repository, local environments, CI, health endpoint, migrations, documentation.	Frontend and API run locally; CI passes.
M1 - Authentication	Sign-up, login, protected routes, onboarding, profile persistence.	New user reaches personalized dashboard.
M2 - Persistent Chat	Conversation/message models, APIs, provider abstraction, SSE streaming, history.	User can resume a streamed conversation after signing out.
M3 - Mentor Mode	Adaptive explanations, quiz action, exercises, learning summaries.	One complete teach-test-practise flow works.
M4 - Interview Engine	Configuration, state machine, questions, follow-ups, structured scoring, report.	Five-question interview produces a useful report.
M5 - Memory	Summaries, confirmed facts,	Later sessions accurately use

Milestone	Deliverable	Exit gate
	weak areas, embeddings, retrieval, memory controls.	approved prior context.
M6 - Team Simulations	Stand-up, blocker discussion, code review disagreement, debrief rubric.	User receives actionable communication feedback.
M7 - Dashboard	Trends, repeated weaknesses, completed sessions, next recommendation.	Metrics reflect evaluated performance, not vanity counts.
M8 - RAG	PDF ingestion, chunking, embeddings, retrieval, citations, deletion.	Answers cite relevant uploaded material.
M9 - Voice	Speech input/output, transcript, filler words, pace and pause analysis.	Voice interview produces technical and speaking feedback.
M10 - GitHub	OAuth, repository selection, safe ingestion, project analysis, project interview.	User can rehearse explaining a real repository.
M11 - Production	Limits, privacy controls, backups, observability, export/delete, billing readiness.	System is safe and supportable for external users.

10. Immediate Execution Sequence

Build in vertical slices. Each slice must leave the application usable and deployable.

Sprint	Primary outcome
Sprint 1	Create monorepo, Next.js app, FastAPI app, PostgreSQL, Docker, CI, health endpoint.
Sprint 2	Add Supabase Auth, profiles, protected dashboard, onboarding.
Sprint 3	Add conversations/messages persistence and non-AI chat interface.
Sprint 4	Add provider abstraction, OpenAI integration, SSE streaming, usage records.
Sprint 5	Add Mentor Mode, quiz and exercise actions.
Sprint 6	Add interview configuration and state machine.
Sprint 7	Add structured evaluation and report history.
Sprint 8	Add basic memory and memory-management UI.
Sprint 9	Add three team simulations and debriefs.
Sprint 10	Add progress dashboard, polish, documentation, public beta deployment.

11. Definition of Done

- Requirements and acceptance criteria are documented.
- Authorization and ownership checks are implemented.
- Database changes include reversible migrations.
- Loading, empty, success, and failure states are handled.

- Changed behavior has appropriate tests.
- Linting, type checking, and tests pass.
- No secrets or sensitive content are logged.
- Documentation and API contracts are updated.
- The feature works in the deployed environment.
- The final diff contains no unrelated changes.

12. Testing and Evaluation

- Unit tests: prompt composition, state transitions, scoring validation, memory selection, permissions.
- Integration tests: API plus database, authentication, persistence, provider failures, retrieval.
- End-to-end tests: onboarding, chat, interview completion, report display.
- AI evaluation set: strong, weak, partially correct, confident-but-wrong, and correct-but-unclear answers.
- Regression tests: store prompt version and model metadata with evaluated outputs.

13. Security and Cost Controls

- Keep provider keys server-side.
- Validate authentication on every protected endpoint.
- Enforce user ownership for every record.
- Sanitize rendered Markdown and prohibit arbitrary HTML execution.
- Treat uploaded files and repository content as untrusted input.
- Add request size, output token, file size, and iteration limits.
- Track model, input tokens, output tokens, estimated cost, feature, and user per request.
- Allow users to inspect and delete saved memories and conversations.
- Use smaller models for extraction/classification when quality is sufficient; reserve larger models for high-value coaching.

14. First 20 GitHub Issues

7. Initialize the monorepo and workspace tooling.
8. Create the Next.js application.
9. Create the FastAPI application.
10. Add local PostgreSQL with Docker Compose.
11. Add environment-variable validation.
12. Configure formatting, linting, and type checking.
13. Add GitHub Actions quality checks.
14. Create the API health endpoint.
15. Configure SQLAlchemy and Alembic.
16. Create profile model and migration.
17. Integrate Supabase authentication.
18. Build the onboarding form and endpoint.
19. Create conversation model and migration.
20. Create message model and migration.

21. Implement conversation CRUD with ownership checks.
22. Build the conversation list and chat shell.
23. Create the internal LLM provider interface.
24. Implement the OpenAI provider and SSE endpoint.
25. Persist complete assistant responses and usage metadata.
26. Add integration tests for authentication, conversations, and streaming.

15. Root Files to Create Next

File	Role
PROJECT_GUIDE.pdf	Long-form product, architecture, and roadmap source of truth.
AGENTS.md	Short operational instructions automatically read by Codex.
README.md	Setup, commands, product summary, and contribution entry point.
docs/product/PRD.md	Detailed requirements and user journeys.
docs/architecture/SYSTEM.md	Architecture diagrams, boundaries, and data flow.
docs/decisions/ADR-001.md	First architecture decision: modular monolith.
.env.example	Documented environment variables without secrets.
Makefile	Consistent setup, lint, test, migration, and run commands.

16. Official Codex References

These links are included because Codex behavior and configuration may evolve. Check the current official documentation before changing repository-level agent conventions.

Codex documentation: <https://developers.openai.com/codex>

Codex CLI: <https://developers.openai.com/codex/cli>

AGENTS.md guidance: <https://developers.openai.com/codex/agent-configuration/agents-md>

Configuration reference: <https://developers.openai.com/codex/config-reference>

Build skills: <https://developers.openai.com/codex/build-skills>

Codex prompting guide:

https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide

17. Start Here

Current target: complete Milestones 0-2.

A user must be able to register, complete onboarding, create a conversation, receive a streamed AI response, and reopen the conversation later.

Do not begin voice, RAG, GitHub ingestion, Redis, microservices, or autonomous multi-agent orchestration before this flow is reliable and deployed.

18. Project Working Agreement

- Keep the main branch deployable.
- One issue should represent one reviewable outcome.
- Use Codex for implementation speed, exploration, tests, refactors, and review - not for silently choosing product direction.
- Record meaningful architecture changes as ADRs before or alongside implementation.
- Prefer clear boring code over premature framework abstractions.
- Review generated migrations, authentication logic, permissions, and billing-related code manually.
- At the end of every sprint, update this guide or the related living Markdown documents when assumptions change.

Version note

This is version 1.0 of the project-root guide. Replace the temporary product label after naming the application, then regenerate the PDF whenever major scope, architecture, or milestone decisions change.